



Implementación de IDS con Machine Learning en redes IoT con dispositivo de Edge computing

Jonatan Esmits Hernández Rubio

Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
Ingeniería de Sistemas y Computación
Bogotá, Colombia
2022

Implementación de IDS con Machine Learning en redes IoT con dispositivo de Edge computing

Jonatan Esmits Hernandez Rubio

Tesis presentada como requisito parcial para optar al título de:
Ingeniero de Sistemas y Computación

Director:

Ph.D. Carlos Andrés Lozano Garzón

Director:

Ph.D. German Adolfo Montoya

Grupo de Investigación:

COMIT

Universidad de los Andes

Departamento de Ingeniería de Sistemas y Computación

Ingeniería de Sistemas y Computación

Bogotá, Colombia

2022

Resumen

El crecimiento exponencial de dispositivos IoT en redes físicas abre una posible brecha de seguridad para estas mismas. En este documento se aborda una implementación de un sistema para la detección de intrusos en una red física con el dispositivo Jetson Nano. Este dispositivo de Edge Computing cuenta con una red neuronal entrenada con ataques de tipo *DoS* y *Fuzzers*. La red neuronal cuenta con dos variantes, una entrenada con PCA y otra con 10 componentes, siendo mejor esta última tanto en tiempo como en porcentaje de predicciones de ataques.

Palabras clave: IoT, Redes neuronales, Jetson, Edge Computing, Machine Learning.

Abstract

The exponential growth of IoT devices in physical networks opens a potential security gap for these networks. This paper addresses the implementation of a system for intrusion detection in a physical network with a Jetson Nano device. This Edge Computing device has a neural network trained with DoS and Fuzzers attacks. The neural network has two variants, one trained with PCA and the other with 10 features, the latter being better both in time and in the percentage of attack prediction.

Keywords: Internet of Things, Neural networks, Edge Computing, machine learning.

Contenido

1. Generalidades del proyecto.....	3
1.1 Objetivos	3
1.1.1 Objetivo general.....	3
1.1.2 Objetivos específicos.....	3
1.2 Marco Teórico	3
1.2.1 Internet de las cosas (IoT)	3
1.2.2 Seguridad en IoT	4
1.2.3 Edge Computing	4
1.2.4 Machine Learning	5
1.2.5 Métricas de rendimiento.....	6
2. Propuesta	9
2.1 Red IoT con Jetson nano y NodeMCUs	9
2.1.1 Scapy y Script para Escuchar paquetes	10
2.1.2 NodeMCU y Raspberry Pi como nodos IoT	14
2.2 Modelo Red Neuronal	15
2.2.1 Preparación de los datos	15
2.2.2 Modelo con 10 Features	16
2.2.2.1 Implementación del modelo	16
2.2.2.2 Modelo Integrado en el pipeline	17
2.2.3 Modelo con PCA.....	19
2.2.3.1 Implementación del modelo	19
2.2.3.2 Modelo Integrado en el pipeline	20
3. Resultados.....	23
3.1.1 Modelo con 10 Features	23
3.1.2 Modelo con PCA.....	27
3.1.3 Comparación de los 2 modelos.....	31
4. Conclusiones y trabajo futuro.....	35
4.1 Conclusiones.....	35
4.2 Trabajo futuro	36
5. Bibliografía	37

Lista de figuras

	Pág.
Figura 2-1:	Diagrama arquitectura de red con dispositivo Jetson nano..... 10
Figura 2-2:	Estructura de un paquete IP..... 11
Figura 2-3:	Función para transformar paquetes en DataFrames. 13
Figura 2-4:	Función que extrae la Información de las tramas recursivamente. 14
Figura 2-5:	Estructura del modelo con 10 Features. 17
Figura 2-6:	Código ejecutado en el Jetson Nano para el modelo con 10 features. ... 18
Figura 2-7:	Estructura del modelo con 6 Features y PCA. 20
Figura 2-8:	Código ejecutado en el Jetson Nano para el modelo con PCA..... 21
Figura 3-1:	Accuracy Modelo 10 Features entrenado con 512 Batch size. 25
Figura 3-2:	Loss Modelo 10 Features entrenado con 512 Batch size. 25
Figura 3-3:	Tiempo para 30 rondas Modelo 10 Features. 26
Figura 3-4:	Promedio de tiempo en predicción para 30 rondas Modelo 10 Features. 27
Figura 3-5:	Accuracy Modelo PCA entrenado con 512 Batch size..... 29
Figura 3-6:	Loss Modelo PCA entrenado con 512 Batch size..... 29
Figura 3-7:	Tiempo para 30 rondas Modelo PCA..... 31
Figura 3-8:	Promedio de tiempo en predicción para 30 rondas Modelo PCA..... 31
Figura 3-9:	Promedio de tiempo en predicción comparación de modelos. 32
Figura 3-10:	Promedio predicciones por ronda..... 33

Lista de tablas

	Pág.
Tabla 2-1: Especificación de las Feature para entrenar el modelo.	16
Tabla 3-1: Configuración de entrenamiento y resultados Modelo 10 Features.	24
Tabla 3-2: Configuración de entrenamiento y resultados Modelo PCA.....	28

Introducción

La tecnología ha traído consigo nuevas formas de conectar a las personas entre ellas y su entorno. Una forma de conexión es la red de dispositivos IoT que permite a cualquier dispositivo con un procesador y conexión a internet poder comunicarse entre sí. Facilitando al usuario realizar ciertas actividades como lo son: la toma de medidas por medio de sensores, el control de elementos del hogar como persianas, alarmas, cámaras de seguridad luces, entre otros. Los sistemas IoT han estado presentes no solo en aspectos básicos como los mencionados. Por el contrario, han sido usados en gran variedad de campos como lo son salud, agricultura, cadenas de suministro, educación, transporte, monitorización de tráfico, entre otros [1].

Los analistas predicen que habrá una sociedad con más dispositivos conectados que personas que viven en este planeta. La Corporación Internacional de Datos (IDC) pronosticó que habría 41.600 millones de dispositivos IoT conectados que producirían 79,4 “zettabytes” (ZB) de datos en 2025 en comparación con la población estimada de 8.100 millones [2] [citado en 1, p.1802]. Con esta numerosa cantidad de dispositivos e información producida, el área posible de ataque se ha incrementado en favor de los hackers. Una razón de esta afirmación es que las redes de IoT se basaron en las redes tradicionales, por lo que las brechas de seguridad también se introdujeron en estas nuevas redes [3].

En las redes IoT vemos una gran variedad de dispositivos y, adicionalmente, una gran cantidad de información producida que necesita ser protegida contra intrusos. Comúnmente, estos dispositivos tienen un poder de computación limitado y poca memoria. Ejemplos de tales dispositivos IoT restringidos incluyen sensores, micrófonos, Smart fridges y Smart lights [4].

Al contrario que otras redes tradicionales, los nodos de IoT son sensibles y se asignan en posiciones sin supervisión manual, así como también cuentan con una baja capacidad y recursos limitados, lo que hace que las vulnerabilidades en seguridad sean bastante problemáticas. Además, el rápido desarrollo y la adopción más amplia de dispositivos IoT en nuestras vidas significa la urgencia de abordar estas amenazas de seguridad lo más pronto posible. Debido a la limitación intrínseca de la capacidad y la velocidad de procesamiento, las contramedidas de seguridad tradicionales no se aplican en este campo [3].

Adicionalmente, existen diversidad de ataques los cuales a su vez requieren soluciones diferentes. De proceder a implementar todas las soluciones se consumiría el poder de cómputo y la batería de los dispositivos, lo cual no es aceptable para dispositivos IoT. Por lo tanto, se requieren soluciones livianas y eficientes que abarquen toda la variedad de ataques [3].

En este documento se abordó la implementación de un sistema para la detección de intrusos (IDS) basado en algoritmos de Machine Learning. Se decidió como enfoque el uso de redes neuronales y se diseñó una arquitectura de red física en la cual se desplegó el modelo de Machine Learning usando como principal componente el dispositivo de Edge Computing Jetson Nano de NVIDIA. Este nodo cuenta con la mejor relación costo desempeño para el procesamiento de la información de la red y la posible detección de ataques, en comparación con la Raspberry PI4 y la Jetson TX2 [5].

1. Generalidades del proyecto

1.1 Objetivos

1.1.1 Objetivo general

Implementar un sistema para la detección de intrusos (IDS) con una técnica de Machine Learning en el contexto de redes de IoT con dispositivos de Edge Computing.

1.1.2 Objetivos específicos

- Identificar los requerimientos necesarios para generar una implementación adecuada a una red de IoT con dispositivo Jetson nano.
- Determinar los dataset y los ataques a detectar por medio de la propuesta.
- Identificar e implementar una técnica de Machine Learning en un escenario de prueba que permita detectar ataques en una red IoT.
- Analizar los resultados teniendo en cuenta las métricas típicas para este tipo de algoritmos, así como en cuanto al tiempo de detección de los ataques.

1.2 Marco Teórico

1.2.1 Internet de las cosas (IoT)

El internet de las cosas o IoT, por sus siglas en inglés, es un conjunto de dispositivos electrónicos que tienen la capacidad de conectarse entre sí o con otros sistemas de manera ubicua y sin restricciones de tiempo o lugar. La gran variedad de equipos que entran en esta categoría va desde Smart Devices, sensores, vehículos, máquinas, hasta humanos o cualquier otro objeto que sea conocedor de su contexto y tenga la capacidad de comunicarse con otra entidad [6].

1.2.2 Seguridad en IoT

La amplia utilización de dispositivos IoT, que van desde sensores de temperatura hasta dispositivos médicos, precisa un enfoque importante en la seguridad de estos dispositivos y redes conformadas por estos. Los principales requerimientos en este tipo de dispositivos incluyen (i) autenticación y rastreo, (ii) integridad de la información, (iv) confianza mutua, (v) privacidad y olvido digital [7].

- (i) Autenticación y rastreo: Miles de millones de dispositivos al formar parte del ecosistema IoT necesitarán un identificador único para que esta infraestructura pueda ser capaz de rastrearlos en cualquier momento. Es importante garantizar que todos los datos recogidos sean auténticos [7].
- (ii) Integridad de la información: Cada usuario debe poder confiar en que los datos suministrados por un dispositivo IoT son fiables, es decir, que han sido recogidos por el sensor especificado en el lugar y el momento indicados [7].
- (iii) Confianza mutua: Recientemente, se han presentado varios planes para asegurar la confianza en el IoT. Estas soluciones, que se basan en primitivas de seguridad de hardware, deben seguir optimizando los costes y la energía. Además, se pueden emplear métodos de atestación de hardware y software para las tareas relacionadas con la confianza. Otro posible problema es la confianza del operador en la cual los sensores y los dispositivos IoT pueden aprobar y confiar en las órdenes de los usuarios de IoT [7].
- (iv) Privacidad y olvido digital: Además de los datos sensoriales que recogerán los dispositivos IoT, habrá enormes conjuntos de datos relacionados con las actividades de comunicación entre varios usuarios y dispositivos IoT. Bajo este escenario entra el olvido digital un mecanismo para borrar la información de un conjunto de datos de manera demostrable [7].

1.2.3 Edge Computing

La computación de borde, o Edge computing, ha hecho su debut histórico como resultado del creciente volumen de datos y la creciente necesidad de procesamiento de datos. Para

el número cada vez mayor de dispositivos terminales y datos, la tecnología Edge computing ofrece capacidades de inteligencia artificial y mejora la estabilidad de los servicios. Esta tecnología se sitúa cerca del origen de los datos, específicamente, en el borde de la red en donde se procesan y almacenan los datos. Ofrece a los usuarios servicios cercanos a la fuente y tiene conocimiento de la proximidad y la ubicación. Procesa los datos con mayor rapidez, en tiempo real y de forma segura. Además, puede ahorrar gastos y aliviar la presión sobre el ancho de banda de la red, de igual manera, resuelve el problema del uso excesivo de energía en la computación en la nube. La gran variedad de aplicaciones del Edge computing abarca sectores como la fabricación, la energía, los hogares inteligentes y el transporte [8].

1.2.4 Machine Learning

Este tipo de algoritmo es un procedimiento informático que utiliza los datos de entrada para completar una tarea propuesta sin estar explícitamente programado (definido el paso a paso) para hacerlo. Estos algoritmos, al ajustar o adaptar automáticamente su diseño como resultado de la repetición (o la experiencia), se pueden considerar que están "programados de forma indirecta" en el sentido de que mejoran cada vez más en la consecución del objetivo [9].

Parte fundamental de esta técnica es el entrenamiento. Este proceso consiste en proveer a la máquina con pequeñas partes de la información de entrada y conjuntamente suministrar la solución esperada. Luego, el algoritmo se configura adecuadamente para no solo poder deducir la respuesta usando los datos de entrenamiento, sino que también se generalice y permita obtener el resultado deseado en todo momento. El componente de "aprendizaje" del machine learning es este entrenamiento. Dicho proceso no tiene por qué restringirse a una primera aproximación durante un periodo de tiempo fijo. Un buen algoritmo puede realizar un aprendizaje "permanente" a medida que analiza datos frescos y se educa con sus fallos al igual que los humanos [9].

Un algoritmo informático puede modificarse a sí mismo de diversas maneras en respuesta al entrenamiento. Mediante la optimización iterativa se pueden cambiar los parámetros numéricos del algoritmo. Este algoritmo se puede organizar en una red de caminos computacionales potenciales para producir los mejores resultados. Otro posible comportamiento que el algoritmo pueda tomar es que, a partir de los datos de entrada, puede generar distribuciones de probabilidad y utilizarlas para hacer predicciones. El ideal

del aprendizaje automático es imitar cómo las personas y otros seres sensibles aprenden a interpretar los datos sensoriales (de entrada) para alcanzar un objetivo [9].

El algoritmo de machine Learning a implementar en este documento es una red neuronal. Las redes neuronales o Artificial Neural Networks (RNA) conforman parte del conjunto de machine learning superficial donde se aprende de información con clases predefinidas. Específicamente, las RNA se consideran modelos no lineales ya que intentan descubrir asociaciones complejas entre los datos de entrada y los de salida. Extrayendo una muestra de los datos, en lugar de considerar todo el conjunto, las redes neuronales reducen el coste y el tiempo requerido para dar un resultado. Ejemplos de algoritmos con esta arquitectura encontramos el Perceptrón, red de back-propagation, entre otros [10].

1.2.5 Métricas de rendimiento

Es importante tener claro cómo se evaluarán los resultados para una tarea de clasificación. Por tanto, se definirán a continuación las métricas escogidas que serán mencionadas durante este documento para evaluar el modelo. En la Tabla 1-1 se presenta la matriz de confusión.

Tabla 1-1: Estructura de una tabla de confusión.

Predicciones	Reales	
	TP	FN
	FP	TN

Comúnmente, en las tareas de clasificación binarias se usa el valor 1 como un valor verdadero y 0 como un valor falso. Para facilitar el lenguaje se remplazará estos términos como “Sí” y “No”, respectivamente. A continuación, se describe cada valor presente en la Tabla 1-1:

- Falsos Positivos (FP): Número de valores que fueron clasificados como “Sí” pero en realidad son “No”.
- Falsos Negativos (FN): Número de valores que fueron clasificados como “No” pero en realidad son “Sí”.

- Verdaderos Positivos (TP): Número de valores que fueron clasificados como “Sí” y realmente son “Sí”.
- Verdaderos Negativos (TN): Número de valores que fueron clasificados como “No” y realmente son “No”.

Conjuntamente se usan las métricas de Loss, Accuracy, Recall, Precision y F1-Score las cuales se definen de la siguiente forma:

- Loss: Es una medida de penalización por clasificación errónea. Todos los algoritmos de aprendizaje, basados en modelos, tienen una función de pérdida y lo que hacemos para encontrar el mejor modelo es intentar minimizar el objetivo conocido como función de coste o loss function [11]. La función que se usará como función loss es la binary crossentropy, la cual está definida por la fórmula 1.1:

$$y_{i,l} \in \{0,1\} \wedge l \in [1,L) \wedge i \in [1,N)$$

$$\text{binary crossentropy} = -(y_{i,l} * \ln(\hat{y}_{i,l}) + (1 - y_{i,l}) * \ln(1 - \hat{y}_{i,l})) \quad (1.1)$$

- Accuracy: Se calcula a partir del número de valores correctamente clasificados dividido por el de valores clasificados [11]. Si usamos los valores de la matriz de confusión en la fórmula 1.2 podemos ver su definición:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (1.2)$$

- Recall: Se calcula a partir del número de valores positivos predichos correctamente, sobre el total de valores positivos del conjunto de prueba [11]. Si usamos los valores de la matriz de confusión en la fórmula 1.3 podemos ver su definición:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (1.3)$$

- Precision: Se calcula a partir del número de valores positivos predichos correctamente, sobre el total de valores positivos predichos [11]. En la fórmula 1.4 podemos encontrar su definición:

$$Precision = \frac{TP}{TP+FP} \quad (1.4)$$

- F1-Score: Puede interpretarse como una media armónica entre Precision y el Recall, donde el F1-Score alcanza su mejor valor en 1 y la peor puntuación en 0 [12]. En la fórmula 1.5 podemos encontrar su definición:

$$F1 - Score = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (1.5)$$

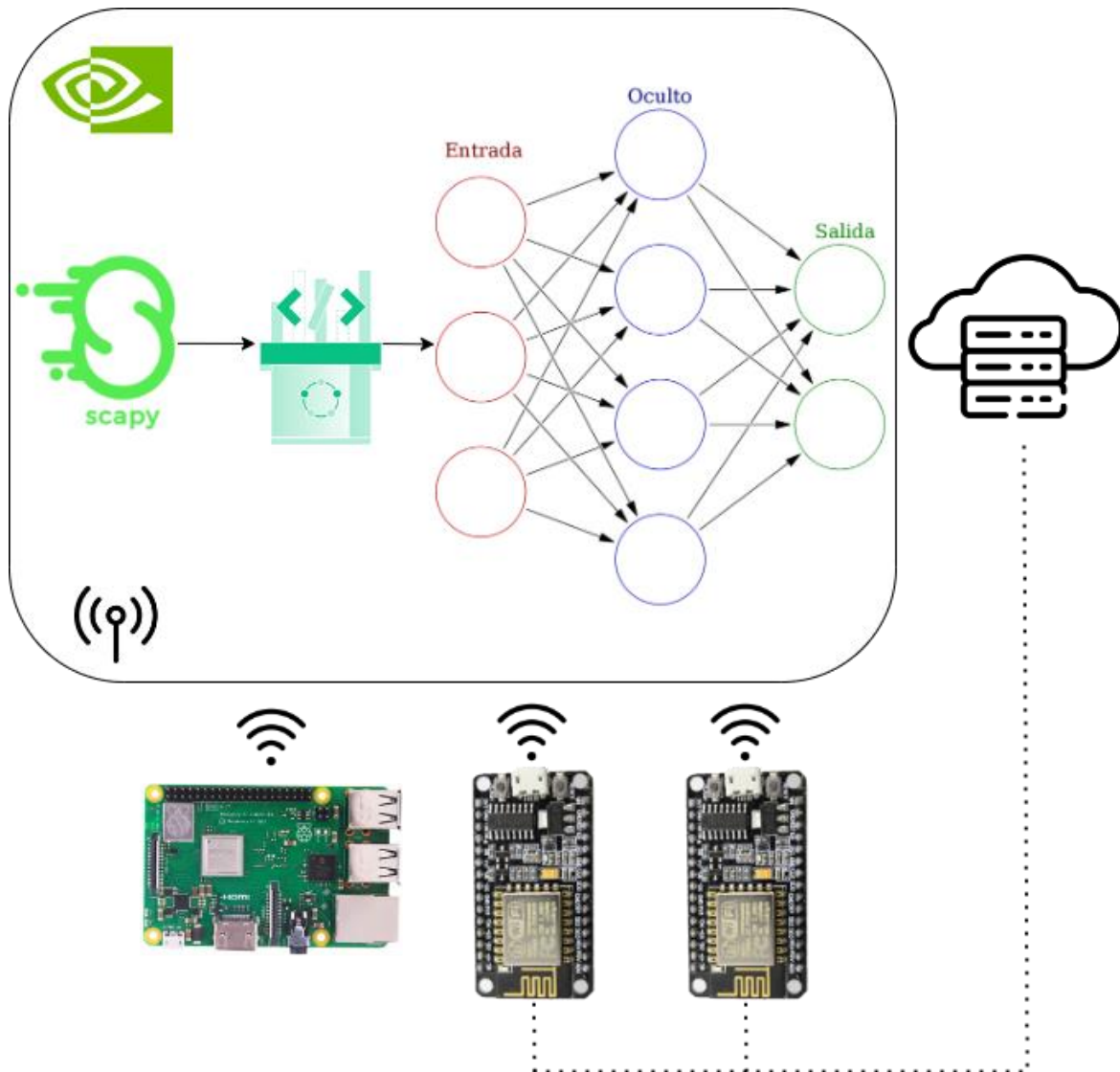
2.Propuesta

Fundamentalmente, la propuesta está dividida en: la definición de la arquitectura de red IoT con dispositivo Jetson nano y como interactúan entre sí los componentes que la conforman; conjuntamente se compone por el entrenamiento de la red neuronal y los pasos requeridos para transformar los datos y llevar a cabo dicho entrenamiento. Estas dos secciones convergen al final para poder presentar un modelo de Machine Learning en producción que intercepte paquetes, los envíe al modelo y este los clasifique como un posible ataque o no.

2.1 Red IoT con Jetson nano y NodeMCUs

En la Figura 2-1 se presenta la arquitectura propuesta para el funcionamiento del experimento realizado. Este diagrama contiene la representación de una Jetson nano que en su interior tiene instalado: (i) El algoritmo de Machine Learning, (ii) el script que transforma el datagrama interceptado por la librería Scapy y (iii) el HostPot para compartir internet a los dispositivos por medio de Wi-Fi. Adicionalmente, se presenta el tipo de comunicación que realizan los dispositivos IoT con el servidor destino el cual es un servidor MQTT que permite publicar mensajes en un tópico definido. En las subsecciones de este capítulo se explica a fondo el porqué de las decisiones tomadas y el desarrollo a detalle de cada apartado.

Figura 2-1: Diagrama arquitectura de red con dispositivo Jetson nano.



Composición del autor.

2.1.1 Scapy y Script para Escuchar paquetes

Como parte del desarrollo del proyecto se realizó un análisis que permitiera identificar la forma más conveniente de interceptar paquetes en tiempo real para ser procesados por modelos de Machine Learning. Se comparó el programa más conocido para la escucha de paquetes de red, Wireshark, con la librería Scapy. Wireshark permite espiar los paquetes de una interfaz de red, por medio de una interfaz de usuario, al mismo tiempo que da la

posibilidad de generar un archivo con los datos recopilados. En contra posición, Scapy ofrece esta y otras características al permitir el desarrollo de estas acciones por medio de un lenguaje de programación conocido como lo es Python. Por lo tanto, el uso de esta librería es parte fundamental para el desarrollo de este proyecto.

Adicionalmente, se requirió del entendimiento de la estructura de un datagrama de red. Este aprendizaje facilitó la realización de las respectivas conversiones de estructuras y permitió una fácil manipulación de los datos. En Figura 2-2 podemos evidenciar como se estructura un paquete. Este paquete es un datagrama de DNS, contiene tanto la información básica de direcciones de origen como de destino, los puertos de origen y puertos de destino. Por consiguiente, el paquete de DNS contiene la petición de nombre de dominio a encontrar de tipo AAAA (mapea nombres de host a la IP del host). En este ejemplo, Google.com.

Figura 2-2: Estructura de un paquete IP.

```

Ether / IP / UDP / DNS Ans "2607:f8b0:4005:807::200e"
IP / UDP / DNS Ans "2607:f8b0:4005:807::200e"
UDP / DNS Ans "2607:f8b0:4005:807::200e"
DNS Ans "2607:f8b0:4005:807::200e"
###[ Ethernet ]###
  dst      = 88:e9:fe:6a:92:52
  src      = 80:37:73:96:9b:db
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x20
  len      = 84
  id       = 58919
  flags    =
  frag     = 0
  ttl      = 122
  proto    = 17
  chksum   = 0x360c
  src      = 84.54.22.33
  dst      = 10.1.10.53
  \options \
###[ UDP ]###
  sport    = 53
  dport    = 53
  len      = 64
  chksum   = 0xfe25
###[ DNS ]###
  id       = 12
  qr       = 1
  opcode   = QUERY
  aa       = 0
  tc       = 0
  rd       = 1
  ra       = 1
  z        = 0
  ad       = 0
  cd       = 0
  rcode    = ok
  qdcount  = 1
  ancount  = 1
  nscount  = 0
  arcount  = 0
  \qd      \
  |###[ DNS Question Record ]###
  |  qname   = 'google.com.'
  |  qtype   = AAAA
  |  qclass  = IN
  \an      \
  |###[ DNS Resource Record ]###
  |  rrname  = 'google.com.'
  |  type    = AAAA
  |  rclass  = IN
  |  ttl     = 299
  |  rdlen   = None
  |  rdata   = 2607:f8b0:4005:807::200e
  ns       = None
  ar       = None

```

Datos del autor.

Dada la distribución de la información de un paquete, en la Figura 2-3 podemos ver el código usado para transformar la información de un paquete en un DataFrame de Pandas, una librería de Python para manejo de datos estructurados. Específicamente en las líneas 3 a la 10 se realiza una conversión de todos los paquetes escuchados a un diccionario de Pandas con la función *'process_until_no_payload'* cuya definición es recursiva y podemos encontrar en la Figura 2-4. Cabe resaltar que las líneas 12 a la 18 de la Figura 2-3 nos permite acomodar las columnas con los nombres y posiciones esperados como entradas de la red neuronal que será descrita en otra sección.

Figura 2-3: Función para transformar paquetes en DataFrames.



```
1 def get_df_from_traces(list_packets):
2     ditionary= {}
3     for i in range(len(list_packets)):
4         packet = list_packets[i]
5
6         if (not (IP in packet and Ether )):
7             continue
8
9         ditionary[i]={ 'time':packet.time, 'len': len(packet[IP][type(packet[IP])).payload)}
10        process_until_no_payload(packet[IP], i, ditionary)
11
12    necessary_columns = ['time', 'len', 'ttl', 'proto', 'sport', 'dport', 'seq', 'window']
13    real_columns = ["sport","dport","proto","len","ttl","window","seq","time"]
14
15    df = pd.DataFrame(ditionary)
16    df= df.transpose()
17    df = df[necessary_columns]
18    df = df.reindex(columns=real_columns)
19    df.dropna(inplace=True)
20    return df
```

Generado por el autor.

Dado que cada paquete de red tiene campos anidados, en algunos casos distintos a los demás tipos de paquetes, en la Figura 2-4 en las líneas de la 32 a la 45 se verifica el tipo de paquete a transformar y se asigna una lista de elementos a buscar definida por la librería Scapy. Luego, se procede a extraer la información iterando sobre la lista de elementos como vemos en las líneas 47 a la 58 Figura 2-4.

Figura 2-4: Función que extrae la Información de las tramas recursivamente.

```

24 def append_dictionary(d1,d2, key):
25     return dict(d1[key] if d1.get(key)!= None else {},**d2 )
26
27 def process_until_no_payload(packet, key, dit):
28     if(packet is None or isinstance(packet, NoPayload)):
29         return None
30
31     fields_to_search=[]
32     if isinstance(packet,IP):
33         fields_to_search=ip_fields
34     elif isinstance(packet,TCP):
35         fields_to_search=tcp_fields
36     elif isinstance(packet,UDP):
37         fields_to_search=udp_fields
38     elif isinstance(packet,DNS):
39         fields_to_search=dns_fields
40     elif isinstance(packet,DNSRR):
41         fields_to_search=dns_sub_fields_rr
42     elif isinstance(packet,DNSQR):
43         fields_to_search=dns_sub_fields_qr
44     elif isinstance(packet,ICMP):
45         fields_to_search=icmp_fields
46
47     for field in fields_to_search:
48         if field == 'options' and field in packet.fields:
49             # Retrieving number of options defined in IP Header
50             dit[key] = append_dictionary(dit,{field:len(packet.fields[field])}, key)
51         elif field in packet.fields:
52             if(isinstance(packet,DNS)):
53                 if(field == 'an' or field == 'qd'):
54                     process_until_no_payload(packet.fields[field],key,dit)
55             else:
56                 dit[key] = append_dictionary(dit,{field:packet.fields[field]}, key)
57             else:
58                 dit[key] = append_dictionary(dit,{field:packet.fields[field]}, key)
59
60     process_until_no_payload(packet.payload,key,dit)

```

Generado por el autor.

2.1.2 NodeMCU y Raspberry Pi como nodos IoT

Se decidió usar el dispositivo NodeMCU ESP8266 modulo Wi-Fi por su facilidad de programar con Arduino y la compatibilidad para la comunicación inalámbrica. Conjuntamente estos nodos se conectan a sensores de temperatura para simular una transmisión de datos constante en la red, de manera que se simule un escenario típico

para estas arquitecturas de red. Adicionalmente, se usó una Raspberry Pi para enviar paquetes maliciosos por la red durante las pruebas con el fin de aproximar un poco más la simulación del ambiente de red, en el que se encontraban los nodos IoT, a un posible escenario real.

2.2 Modelo Red Neuronal

Se escogió el dataset UNSW-NB15 [13] ya que contiene tramas de red diferenciadas por tipo de ataque y segmentación de toda la información que estas poseen como protocolos, tiempos, direcciones IP, entre otros. Adicionalmente, contiene columnas calculadas basadas en la información del conjunto de datos.

Dada la información anterior se propone el entrenamiento de dos modelos de redes neuronales. El primero con 10 parámetros escogidos del dataset [13]. El Segundo modelo entrenado con la técnica de “*principal component analysis*” PCA a partir de los mismos parámetros que el modelo 1. En las siguientes secciones se describirá mejor el proceso llevado a cabo.

2.2.1 Preparación de los datos

Los datos de [13] estaban separados en varios archivos ‘.csv’ los cuales fueron leídos y agrupados en un DataFrame de Pandas. Posteriormente, se escogieron las columnas que contuviesen las features o parámetros de la Tabla 2-1. Estas features fueron la intersección entre la información presente en el dataset y los datos obtenidos por la librería Scapy.

Acto seguido, se procedió a filtrar por tipos de tramas de red las cuales fueron: ‘DoS’, ‘Fuzzers’ y ‘Normal’. Esto con el fin de poder replicar los tipos de ataques en la fase de pruebas. Posteriormente, se aplicó un scaler para transformar los datos y dejarlos entre el rango de 0 y 1 con la función ‘MinMaxScaler’ de *sklearn*. Luego, se dividieron los datos en conjunto de validación y conjunto de entrenamiento con una proporción de 15% y 75% respectivamente. Adicionalmente, los datos nulos e inválidos se filtraron y se redujo la cantidad de datos con etiqueta ‘Normal’, para que los modelos tuviesen una proporción similar con respecto a los otros tipos de tramas de red. Este paso fue realizado con la función ‘RandomUnderSampler’ de ‘imblearn’.

Asimismo, se aplicó ‘SMOTENC’ para hacer sobre muestreo a los datos y que todas las etiquetas queden con el mismo número de filas. Estos procesos de sobre muestreo y submuestreo tuvieron el objetivo de igualar la cantidad de filas por cada tipo de etiqueta de los dos conjuntos de datos (el de entrenamiento y el de validación por separado). Finalmente, como preparación de los datos en general, se realizó la obtención de variables ‘dummy’. Este proceso permite obtener variables binarias para indicar la pertenencia o no a una categoría, específicamente de la etiqueta ‘proto’.

Tabla 2-1: Especificación de las Feature para entrenar el modelo.

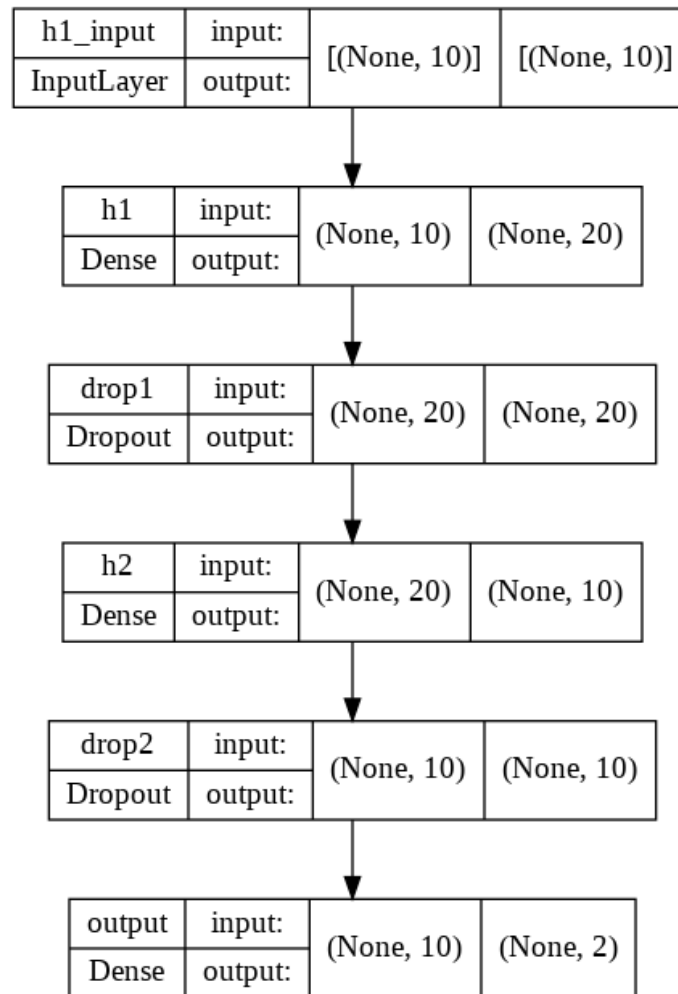
Nombre de la Feature	Descripción
time	Duración de llegada del paquete
len	Tamaño del paquete
TTL	Tiempo de vida del paquete
proto_tcp, proto_udp, proto_dns	Variable dummy para indicar el tipo del protocolo del paquete
sport	Puerto origen
dport	Puerto destino
seq	Numero de secuencia del paquete
window	Tamaño de la Ventana

2.2.2 Modelo con 10 Features

Este modelo fue propuesto como principal aproximación a la clasificación de un ataque dados los parámetros o Features disponibles y mencionados anteriormente.

2.2.2.1 Implementación del modelo

Se propone una red neuronal con una capa de entrada con 10 parámetros los cuales serían los mismos definidos en la Tabla 2-1 Posteriormente, se encuentran 2 capas intermedias, la primera de 20 nodos y la segunda de 10 nodos. Adicionalmente, entre cada capa se usa una capa de dropout para que el modelo no tenga sobreajuste. Por último, la capa final es una capa con 2 salidas para indicar la probabilidad de que un dato corresponda a un ataque o no. En la Figura 2-5 podemos ver la arquitectura final del modelo.

Figura 2-5: Estructura del Modelo con 10 Features.

Generado por el autor.

2.2.2.2 Modelo Integrado en el pipeline

En el Dispositivo Jetson se instalaron las librerías *'tensorflow'*, *'Scapy'*, *'pandas'*, *'sklearn'* y *'numpy'* para poder ejecutar el modelo entrenado y preparar los datos antes de pasarlos a este. En la Figura 2-6 vemos el código que se ejecutó en la Jetson nano. Se usó un *'MinMaxScaler'* (línea 116) para transformar los datos al rango de 0 y 1, al igual que en la etapa de entrenamiento del modelo. En la línea 90 definimos la cantidad de paquetes a escuchar y este será referenciado en este documento como “número de paquetes”.

Figura 2-6: Código ejecutado en el Jetson Nano para el modelo con 10 features.

```
85 model = models.load_model('./models/save_model')
86
87 while True:
88     start_time = time.time()
89
90     packets = sniff(count=200)
91     map_index_protocols = {1: 'icmp', 6: 'tcp', 17: 'udp'}
92
93     data_df_packets = get_df_from_traces(packets)
94
95     map_index_protocols = {1: 'icmp', 6: 'tcp', 17: 'udp'}
96     data_df_packets.loc[data_df_packets["proto"] == 6, "proto"] = map_index_protocols[6]
97     data_df_packets.loc[data_df_packets["proto"] == 1, "proto"] = map_index_protocols[1]
98     data_df_packets.loc[data_df_packets["proto"] == 17, "proto"] = map_index_protocols[17]
99
100    list_dummy_values = []
101    for i in data_df_packets['ttl']:
102        list_dummy_values.append(0)
103
104    if 'icmp' not in data_df_packets['proto'].values:
105        data_df_packets['proto_icmp'] = list_dummy_values
106
107    if 'tcp' not in data_df_packets['proto'].values:
108        data_df_packets['proto_tcp'] = list_dummy_values
109
110    if 'udp' not in data_df_packets['proto'].values:
111        data_df_packets['proto_udp'] = list_dummy_values
112
113
114    data_df_packets = pd.get_dummies(data_df_packets, columns=['proto'])
115
116    scaler = MinMaxScaler(feature_range=(0,1))
117    data_df_packets[['sport', 'dport', 'len', 'ttl', 'window', 'seq', 'time']] = scaler
118    .fit_transform(data_df_packets[['sport', 'dport', 'len', 'ttl', 'window', 'seq', 'time']])
119    for column in data_df_packets.columns:
120        data_df_packets[column] = data_df_packets[column].astype(float)
121
122    data_df_packets.dropna(inplace=True)
123
124    f = model.predict(data_df_packets)
125    print('Network packets clasification:')
126    print(f)
127    response = np.where(f[:,0] >= 0.5, 0, 1)
128    print(response)
129    print('Nos atacan' if np.mean(response) > 0.5 else 'status:OK')
130    print('Time used', time.time()-start_time)
```

Generado por el autor.

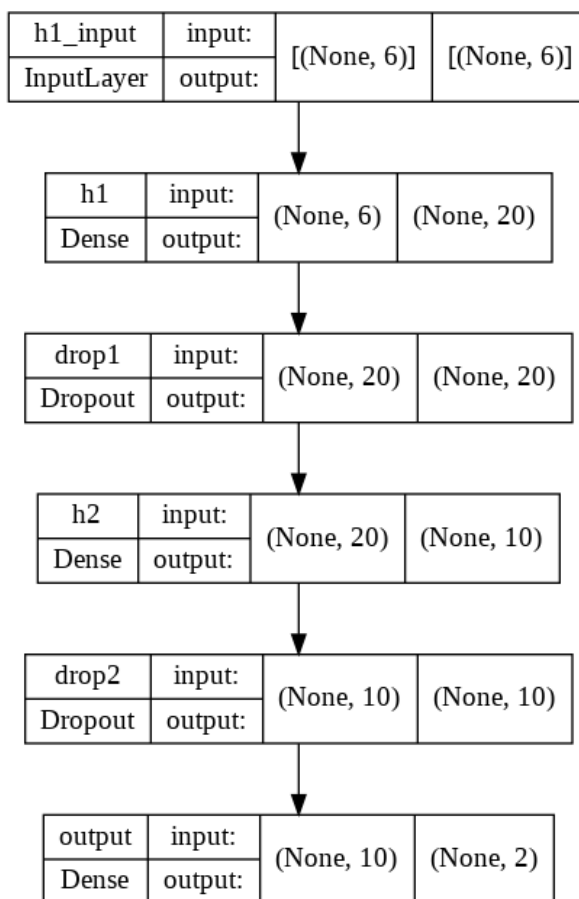
2.2.3 Modelo con PCA

Este modelo fue propuesto para comparar la eficiencia y precisión con respecto al modelo de 10 parámetros. La variación con respecto al modelo anterior es que los parámetros de esta red neuronal fueron escogidos con la técnica de '*Principal Component Analysis*' PCA. Esta técnica permite reducir la dimensionalidad del conjunto de datos aumentando la interpretabilidad a la vez que se minimiza la pérdida de información [14].

2.2.3.1 Implementación del modelo

Se propone una red neuronal con una capa de entrada con 6 parámetros los cuales serían el resultado del PCA descrito anteriormente. Seguidamente, se encuentran 2 capas intermedias, la primera de 20 nodos y la segunda de 10 nodos. Adicionalmente entre cada capa se usa una capa de dropout para que el modelo no tenga sobreajuste. Por último, la capa final es una capa con 2 salidas para indicar la probabilidad de que un dato corresponda a un ataque o no. En la Figura 2-7 podemos ver la arquitectura final del modelo.

Figura 2-7: Estructura del Modelo con 6 Features y PCA.



Generado por el autor.

2.2.3.2 Modelo Integrado en el pipeline

En el Dispositivo Jetson se instalaron las librerías mencionadas en la sección anterior para poder ejecutar el modelo entrenado y preparar los datos antes de ser pasados a este. En la Figura 2-8 vemos el código que se ejecutó en la Jetson nano. Se usó un *MinMaxScaler* (línea 120) para transformar los datos al rango de 0 y 1, al igual que en la etapa de entrenamiento del modelo. Adicionalmente, se usó el mismo transformador PCA ajustado a los datos de entrenamiento para obtener los 6 valores propios que se le pasaron al modelo, esto lo podemos ver en las líneas 88, 89 y 129.

Figura 2-8: Código ejecutado en el Jetson Nano para el modelo con PCA.

```
86 model = models.load_model('./models_pca/save_model')
87
88 with open('./scalers/transformer_pca.pkl', 'rb') as f:
89     pca_loaded = pickle.load(f)
90
91 while True:
92     start_time = time.time()
93
94     packets = sniff(count=200)
95     map_index_protocols = {1: 'icmp', 6: 'tcp', 17: 'udp'}
96
97     data_df_packets = get_df_from_traces(packets)
98
99     map_index_protocols = {1: 'icmp', 6: 'tcp', 17: 'udp'}
100     data_df_packets.loc[data_df_packets["proto"] == 6, "proto"] = map_index_protocols[6]
101     data_df_packets.loc[data_df_packets["proto"] == 1, "proto"] = map_index_protocols[1]
102     data_df_packets.loc[data_df_packets["proto"] == 17, "proto"] = map_index_protocols[17]
103
104     list_dummy_values = []
105     for i in data_df_packets['ttl']:
106         list_dummy_values.append(0)
107
108     if 'icmp' not in data_df_packets['proto'].values:
109         data_df_packets['proto_icmp'] = list_dummy_values
110
111     if 'tcp' not in data_df_packets['proto'].values:
112         data_df_packets['proto_tcp'] = list_dummy_values
113
114     if 'udp' not in data_df_packets['proto'].values:
115         data_df_packets['proto_udp'] = list_dummy_values
116
117
118     data_df_packets = pd.get_dummies(data_df_packets, columns=['proto'])
119
120     scaler = MinMaxScaler(feature_range=(0,1))
121
122     data_df_packets[['sport', 'dport', 'len', 'ttl', 'window', 'seq', 'time']] = scaler
123     .fit_transform(data_df_packets[['sport', 'dport', 'len', 'ttl', 'window', 'seq', 'time']])
124
125     for column in data_df_packets.columns:
126         data_df_packets[column] = data_df_packets[column].astype(float)
127
128     data_df_packets.dropna(inplace=True)
129     data_pca = pca_loaded.transform(data_df_packets)
130
131     f = model.predict(data_pca)
132     print('Network packets clasification:')
133     print(f)
134     response = np.where(f[:,0] >= 0.5, 0, 1)
135     print('Nos atacan' if np.mean(response) > 0.3 else 'status:OK')
136     print('Time used', time.time() - start_time)
```

Generado por el autor.

3.Resultados

3.1.1 Modelo con 10 Features

Durante el proceso de entrenamiento se hicieron pruebas para modelos con la misma arquitectura propuesta, pero con variación en el número de muestras que se propagan por la red neuronal (Batch Size). En la Tabla 3-1 podemos ver las configuraciones y resultados de estas mismas. Adicionalmente, se usaron matrices de confusión para determinar el mejor modelo a usar en el pipeline que correría en la Jetson nano. La mejor combinación obtenida fue para el modelo con 400 épocas y 512 batch size teniendo en cuenta el loss, el accuracy, y la relación entre falsos positivos y falsos negativos.

Cabe resaltar la comparación de los resultados de Precision, Recall y F1-Score para las configuraciones de pruebas que se encuentran en la Tabla 3-1.

Precision: Se obtuvo aumento de su valor si revisamos que, para un batch size de 128 este resultado es 0.089 comparado con un batch size de 512 cuyo resultado es de 0.3836. Este comportamiento quiere decir que el modelo tiende a no etiquetar erróneamente como positivas las muestras que son negativas en la medida que aumenta su batch size.

Recall: Se encuentra una disminución de su valor comparando los batch size de 128 (recall:0.9981) con el batch size de 512 (recall: 0.9968). Este comportamiento quiere decir que el modelo tendió a encontrar en menor medida los valores positivos del conjunto de validación.

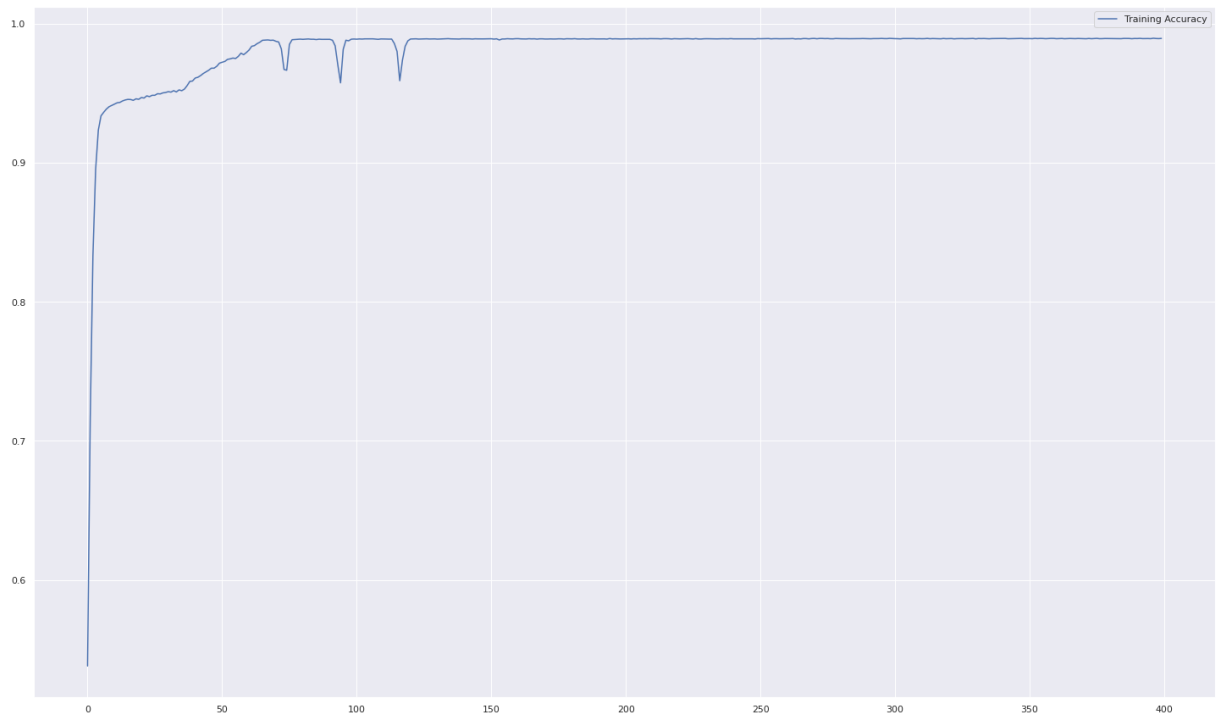
F1-Score: Si analizamos el valor de esta medida, vemos un aumento cuando se pasa de 0.16440 (batch size 128) a 0.55406 (batch size 512). Este resultado significa que hay una tendencia a la mejoría del modelo a la vez que aumenta el batch size.

Tabla 3-1: Configuración de entrenamiento y resultados Modelo 10 Features.

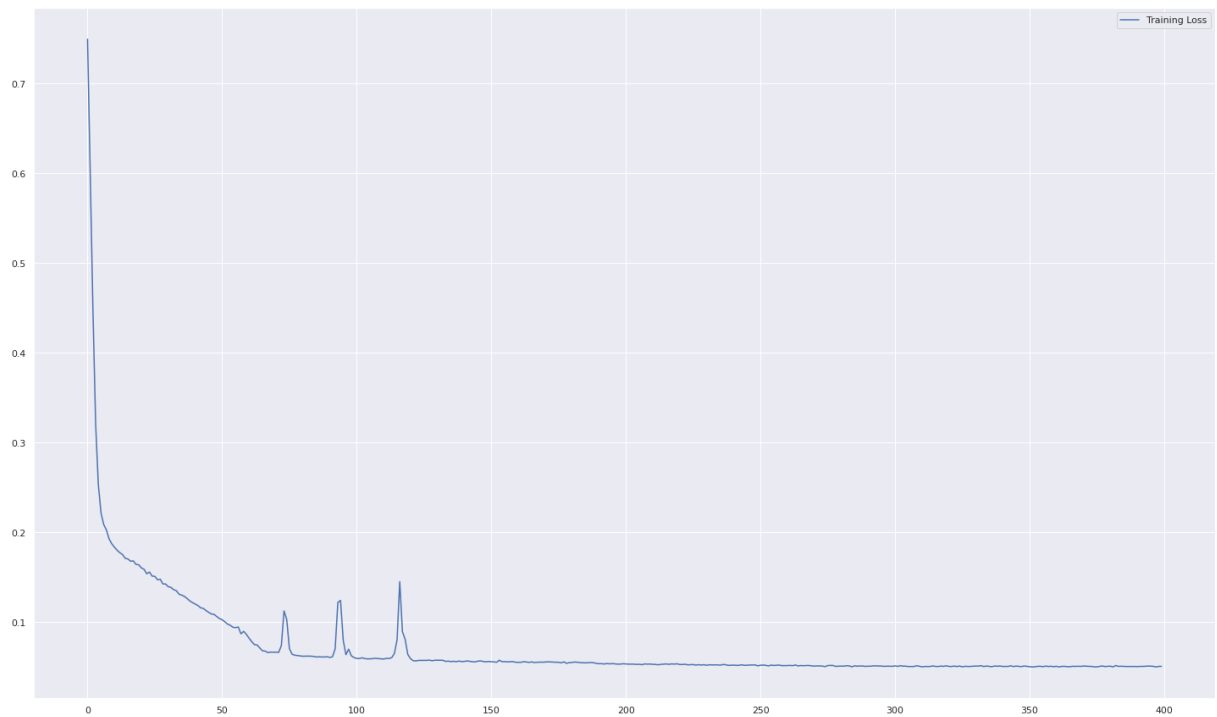
Batch Size	128	256	512
Accuracy	0,8857	0,9819	0,9819
Loss	0,3383	0,103	0,0763
F1-Score	0,1644045	0,5535740	0,5540640
Precision	0,0895796	0,3831871	0,3836567
Recall	0,9981422	0,9968152	0,9968152
Falsos Positivos	38224	6046	6034
Falsos Negativos	7	12	12
Verdaderos Positivos	3761	3756	3756
Verdaderos Negativos	292385	324573	324585

Generado por el autor.

En las figuras Figura 3-1 y Figura 3-2 podemos ver el comportamiento a lo largo de las épocas del accuracy y el los, respectivamente, para el mejor modelo. Como observamos en dichas figuras hasta la época 130 aproximadamente los valores van variando, pero después a estas épocas vemos un comportamiento constante del accuracy cercano al 98,1% (Figura 3-1) y alrededor del 7,6% para el loss (Figura 3-2).

Figura 3-1: Accuracy Modelo 10 Features entrenado con 512 Batch size.

Generado por el autor.

Figura 3-2: Loss Modelo 10 Features entrenado con 512 Batch size.

Generado por el autor.

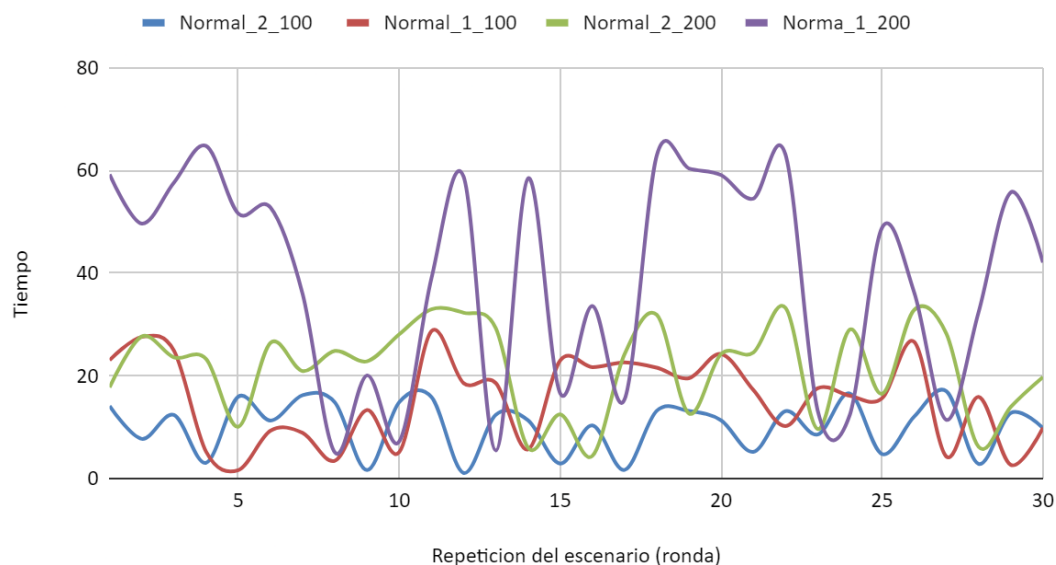
Durante las pruebas del pipeline en el dispositivo Jetson se plantearon los siguientes escenarios con sus respectivos resultados. Se definió escuchar 100 paquetes de red mientras 1 nodo transmitía datos hacia el servidor MQTT por medio de la interfaz expuesta por el dispositivo Jetson (Normal_100). Luego, aumentar la cantidad de paquetes a escuchar con la misma cantidad de nodos (Normal_200). Posteriormente, incrementar la cantidad de nodos a 2 y variar la cantidad de paquetes escuchados de la misma forma que cuando se usaba un solo nodo.

Una vez se obtuvieron los resultados de tiempo para las configuraciones de 100 y 200 paquetes tanto con 1 Nodo como con 2 Nodos, se graficaron estos datos y se obtuvo la Figura 3-3. En esta gráfica se evidencia un comportamiento variado con respecto al tiempo para cada configuración, lo cual dificulta interpretar adecuadamente los datos, por lo que se procede a realizar un promedio de los datos y graficarlos dando como resultado la Figura 3-4.

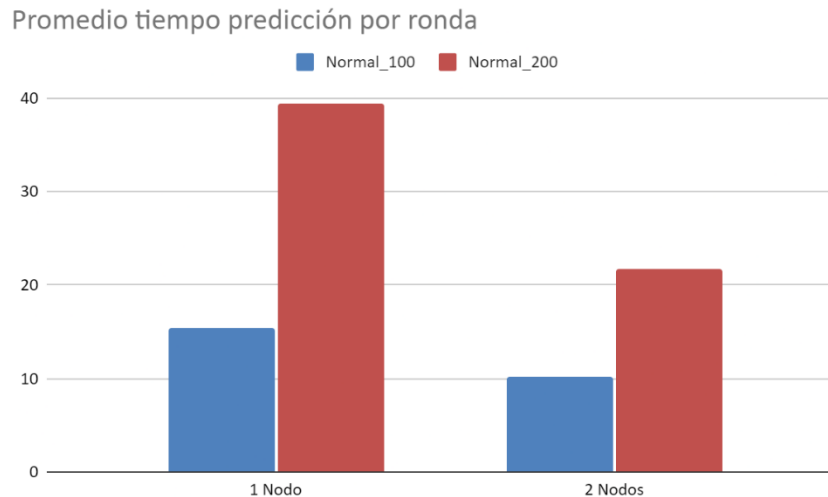
En la Figura 3-4 vemos como para cada escenario el promedio de tiempo tomado en clasificar un segmento de 100 o 200 paquetes como un ataque fue mayor cuando el número de paquetes era 200, esto debido a que la librería Scapy tenía que esperar hasta efectivamente contar 200 paquetes.

Figura 3-3: Tiempo para 30 rondas Modelo 10 Features.

Tiempo para modelo 10 Features



Generado por el autor.

Figura 3-4: Promedio de tiempo en predicción para 30 rondas Modelo 10 Features.

Generado por el autor.

3.1.2 Modelo con PCA

Durante el proceso de entrenamiento se hicieron pruebas para modelos con la misma arquitectura propuesta, pero con variación en el número de muestras que se propagan por la red neuronal (Batch Size). En la Tabla 3-1 podemos ver las configuraciones y resultados de estas mismas. Adicionalmente, se usaron matrices de confusión para determinar el mejor modelo a usar en el pipeline que correría en la Jetson nano. La mejor combinación obtenida fue para el modelo con 400 épocas y 512 batch size teniendo en cuenta el loss, el accuracy, y la relación entre falsos positivos y falsos negativos.

Es importante la comparación de los resultados de Precision, Recall y F1-Score para las configuraciones de pruebas que se encuentran en la Tabla 3-1.

Precision: Se obtuvo aumento de su valor si revisamos que, para un batch size de 128 este resultado es 0.350 comparado con un batch size de 512 cuyo resultado es de 0.382. Este comportamiento quiere decir que el modelo tiende a no etiquetar erróneamente como positivas las muestras que son negativas en la medida que aumenta su batch size.

Recall: Se encuentra un aumento de su valor comparando los batch size de 128 (recall:0.995) con el batch size de 512 (recall: 0.9965). Este comportamiento quiere decir que el modelo tendió a encontrar en mayor medida los valores positivos.

F1-Score: Si analizamos el valor de esta medida, vemos un aumento cuando se pasa de 0.518 (batch size 128) a 0.552 (batch size 512). Este resultado significa que hay una tendencia a la mejoría del modelo a la medida que aumenta el batch size.

Tabla 3-2: Configuración de entrenamiento y resultados Modelo PCA.

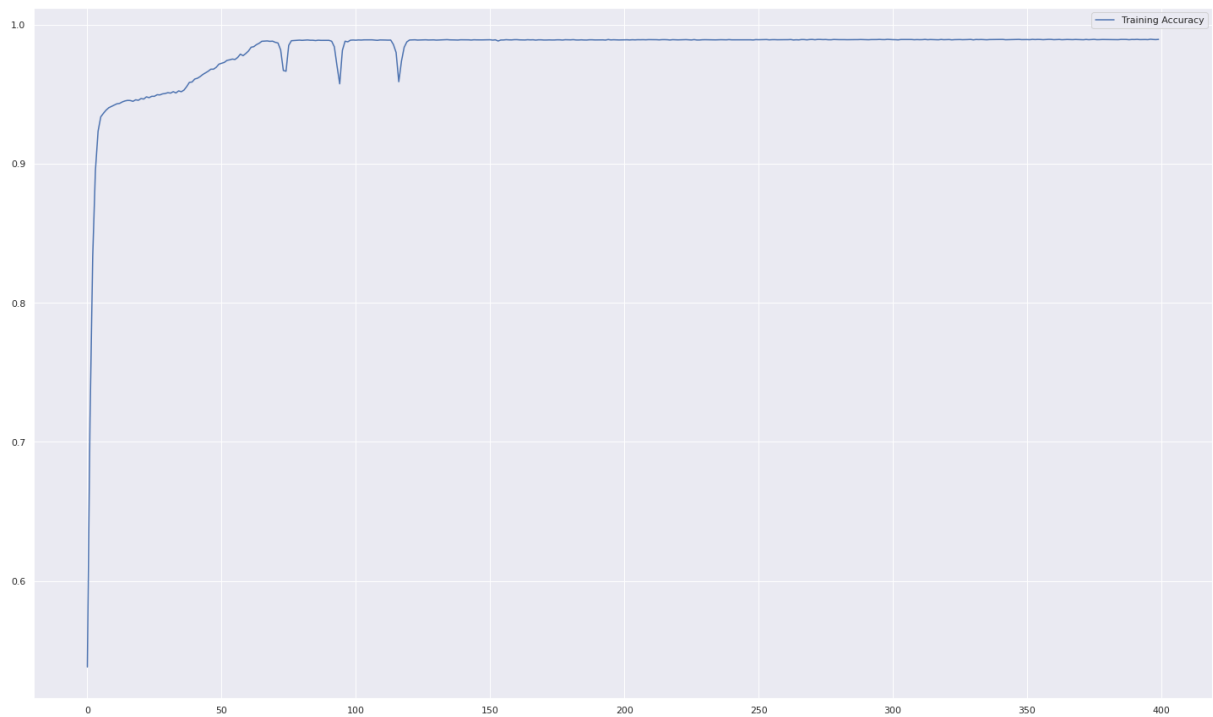
Batch Size	128	256	512
Accuracy	0,9789	0,9816	0,9816
Loss	0,1464	0,0979	0,073
F1-Score	0,5182456	0,5521098	0,5524003
Precision	0,3502996	0,3818949	0,3820958
Recall	0,9955450	0,9960691	0,9965932
Falsos Positivos	7046	6152	6150
Falsos Negativos	17	15	13
Verdaderos Positivos	3799	3801	3803
Verdaderos Negativos	323548	324442	324444

Generado por el autor.

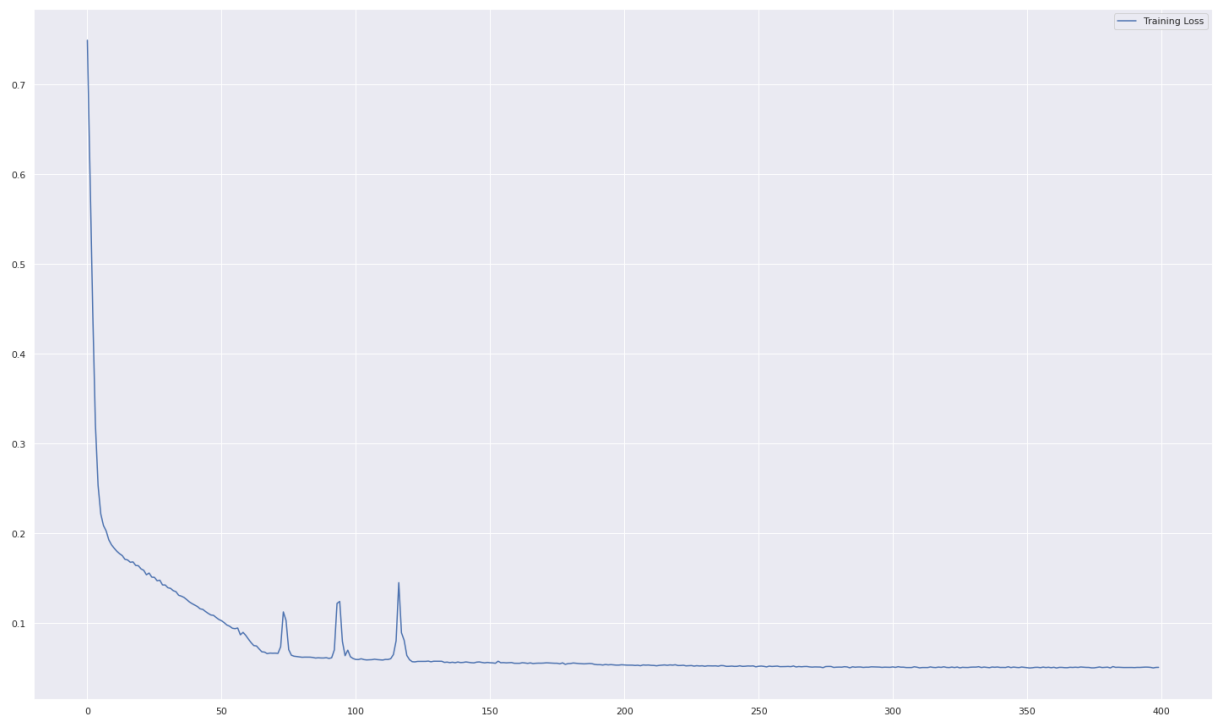
En las figuras

Figura 3-5 y Figura 3-6 podemos ver el comportamiento a lo largo de las épocas del accuracy y el loss respectivamente para el mejor modelo. Como observamos en dichas figuras hasta la época 130 aproximadamente. los valores van variando, pero después a estas épocas vemos un comportamiento constante del accuracy cercano al 98,1% (

Figura 3-5) y para el loss alrededor del 7,3% (Figura 3-6).

Figura 3-5: Accuracy Modelo PCA entrenado con 512 Batch size.

Generado por el autor.

Figura 3-6: Loss Modelo PCA entrenado con 512 Batch size.

Generado por el autor.

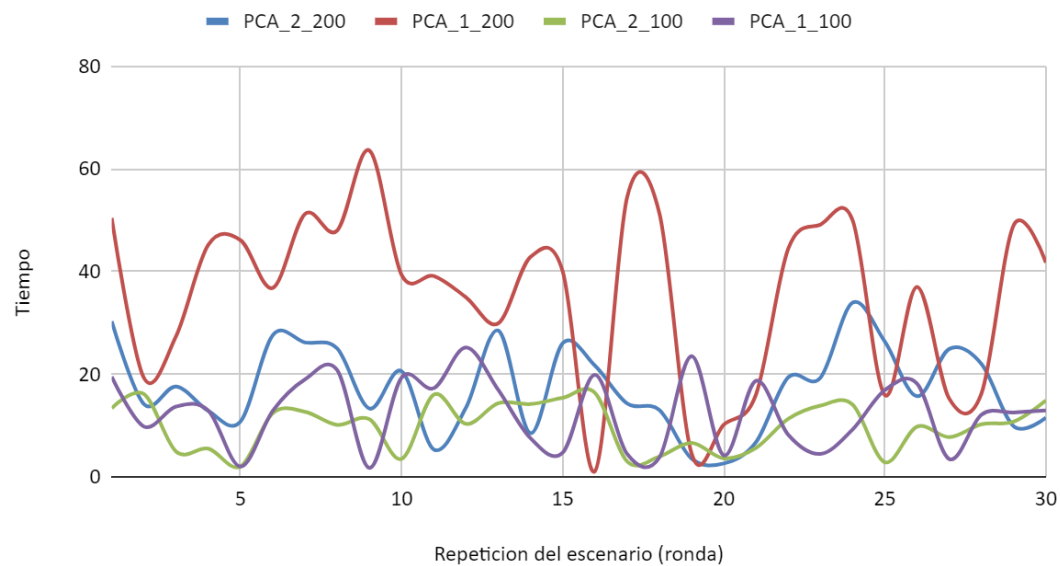
Durante las pruebas del pipeline en el dispositivo Jetson se plantearon los siguientes escenarios con sus respectivos resultados. Se definió escuchar 100 paquetes de red mientras 1 nodo transmitía datos hacia el servidor MQTT, por medio de la interfaz expuesta por el dispositivo Jetson (PCA_100). Luego, aumentar la cantidad de paquetes a escuchar con la misma cantidad de nodos (PCA_200). Posteriormente, incrementar la cantidad de nodos a 2 y variar la cantidad de paquetes escuchados de la misma forma que cuando se usaba un solo nodo.

Una vez se obtuvieron los resultados de tiempo para las configuraciones de 100 y 200 paquetes tanto con 1 Nodo como con 2 Nodos, se graficaron estos datos y se obtuvo la Figura 3-7. En esta gráfica se evidencia un comportamiento variado con respecto al tiempo para cada configuración, lo cual dificulta interpretar adecuadamente los datos, por lo que se procede a realizar un promedio de los datos y graficarlos dando como resultado la Figura 3-8.

En la Figura 3-8 vemos cómo para cada escenario el promedio de tiempo tomado en clasificar un segmento de 100 o 200 paquetes como un ataque fue mayor cuando el número de paquetes era 200, esto debido a que la librería Scapy tenía que esperar hasta efectivamente contar 200 paquetes. Sin embargo, al aumentar el número de nodos de 100 a 200 vemos una reducción en el tiempo promedio para clasificar un paquete.

Figura 3-7: Tiempo para 30 rondas Modelo PCA

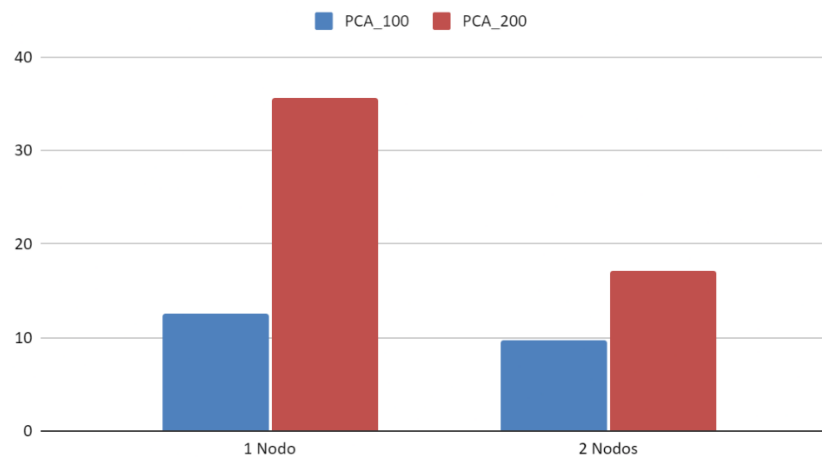
Tiempo para modelo PCA



Generado por el autor.

Figura 3-8: Promedio de tiempo en predicción para 30 rondas Modelo PCA.

Promedio tiempo predicción por ronda



Generado por el autor.

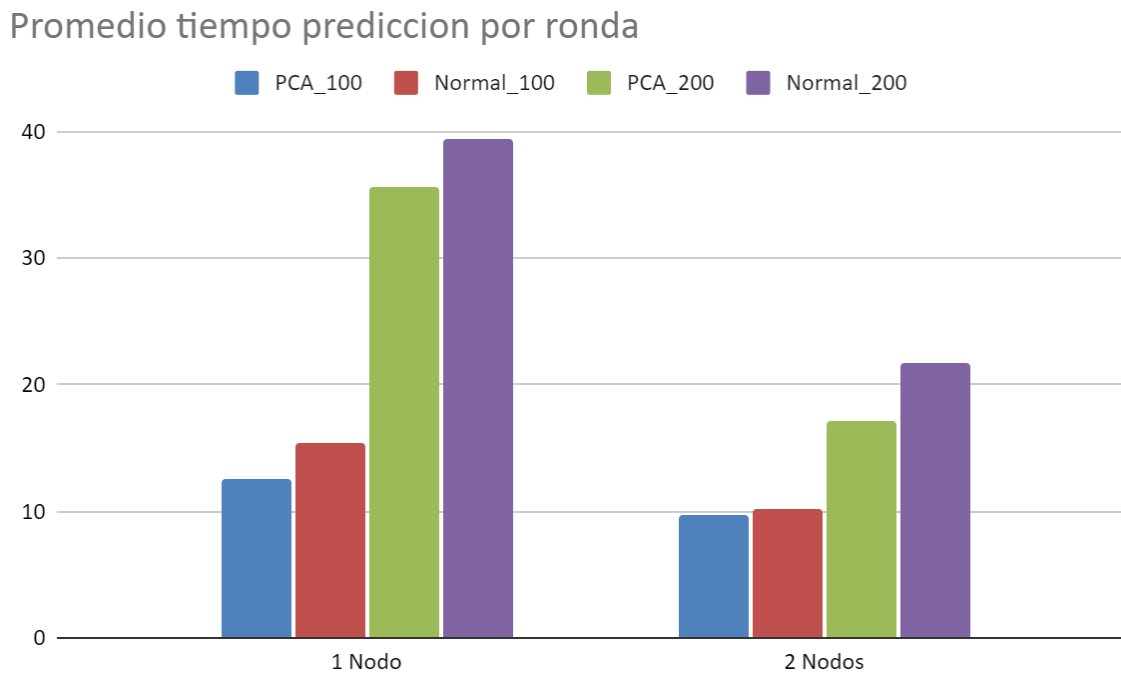
3.1.3 Comparación de los 2 modelos

Si comparamos el promedio de tiempo tomado por cada repetición del escenario para el Modelo con 10 Features (Normal), contra el modelo de 6 Features (obtenidas con PCA),

obtenemos la Figura 3-9. Vemos como incrementa el tiempo, en proporción, cuando la librería tiene que escuchar más paquetes en la red. Adicionalmente, se evidencia un aumento de tiempo al momento de dar una respuesta el modelo cuando no hay suficientes paquetes siendo transmitidos por la red.

Con la Figura 3-10 podemos identificar que, para el Modelo con 10 Features, tanto en el escenario de 100 paquetes con 1 Nodo y 2 Nodos existe una tendencia en clasificar los paquetes que fueron un ataque con un acierto de 61% y 50% respectivamente.

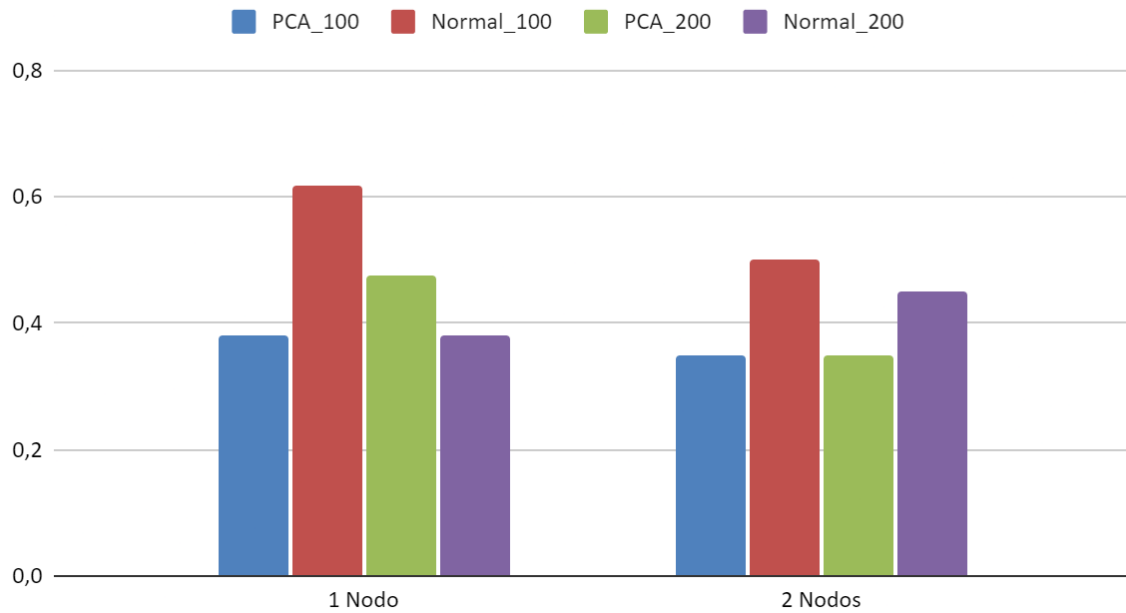
Figura 3-9: Promedio de tiempo (segundos) en predicción comparación de modelos.



Generado por el autor.

Figura 3-10: Promedio predicciones por ronda.

Promedio predicciones por ronda



Generado por el autor.

4. Conclusiones y trabajo futuro

4.1 Conclusiones

Con la creciente adopción de los dispositivos IoT y su gran variedad tanto en diseño, funcionamiento y restricciones, las vulnerabilidades de las redes van también en aumento. Por tanto, acciones para aportar al manejo y protección contra amenazas en estas redes aprovechando nuevos paradigmas, como lo son el machine Learning y el Edge computing, deben ser tomadas.

Como resultado de este trabajo, se implementó un modelo de Machine Learning en una red IoT con dispositivo Edge como Gateway. Esta arquitectura permitió un análisis de paquetes en semi tiempo real, es decir, con cantidad de definida de paquetes a escuchar. Mas detalladamente, se tomaron un número fijo de paquetes espiados en la red y se pasaron a un modelo de Machine Learning para que este clasificara si algún paquete era un ataque o no.

Basado en los resultados descritos para cada configuración de modelo (el primero entrenado con 10 *features*, sección 3.1.1 y el segundo, con la técnica *PCA* de 6 *features*, sección 3.1.2). Podemos concluir que luego de cierto número de épocas de entrenamiento para un *batch* size alto el porcentaje de *loss* se mantiene casi constante, como podemos ver en las figuras: Figura 3-1, Figura 3-2,

Figura 3-5 y Figura 3-6.

Finalmente, como se observan en los resultados, se destaca que la red neuronal entrenada con *PCA* y 6 *features*, tiene mejores métricas en tiempo y en promedio de predicciones en comparación con la red con 10 *features*.

4.2 Trabajo futuro

Teniendo en cuenta las restricciones de tiempo, este sería la primera parte de proyectos futuros que quieran ahondar más en esta temática utilizando la arquitectura propuesta. Se podrían implementar diferentes tipos de ataques, variaciones en la cantidad de nodos y lectura de paquetes. Adicionalmente, se puede revisar la factibilidad de procesar paquetes en tiempo real de forma secuencial usando otras funciones de la librería Scapy.

Conjuntamente, se puede proponer otra arquitectura de red a usar con los mismos escenarios y comparar los resultados obtenidos con los contenidos en este documento. Finalmente, se pueden desarrollar medidas para la mitigación de ataques de redes IoT usando la arquitectura propuesta validando la efectividad de estas y cómo afectan el desempeño comparado con lo obtenido en este proyecto.

5. Bibliografía

- [1] N. Islam, F. Farhin, I. Sultana y M. S. Kaiser, «Towards Machine Learning Based Intrusion Detection in IoT Networks,» *Computers, Materials & Continua*, vol. 69, nº 2, p. 1801–1821, 2021.
- [2] F. H. Atlam y G. B. Willis, «IoT security, privacy, safety and ethics,» *Digital Twin Technologies and Smart Cities*, p. 123–149, 2020.
- [3] J. Deogirikar y A. Vidhate, «Security Attacks in IoT: A Survey,» de *International conference on I-SMAC (IoT in Social, Mobile, Analytics, and Cloud)*, 2017.
- [4] S. Murshed, C. Murphy, D. Hou, N. Khan, G. Ananthanarayanan y F. Hussain, «Machine Learning at the Network Edge: A Survey,» *ACM Computing Surveys*, vol. 54, nº 8, pp. 1-37, 2021.
- [5] A. A. Süzen, B. Duman y B. Şen, «Benchmark Analysis of Jetson TX2, Jetson Nano and Raspberry PI using Deep-CNN,» de *2020 International Congress on Human-Computer Interaction, Optimization, and Robotic Applications*, Ankara, 2020.
- [6] F. Khodadadi, A. Vahid Dastjerdi y R. Buyya, *Internet of Things Principles and Paradigms*, Morgan Kaufmann, 2016, pp. 3-27.
- [7] T. Xu, J. B. Wendt y M. Potkonjak, «Security of IoT systems: Design challenges and opportunities,» de *IEEE International Conference on Computer-Aided Design*, San Jose, CA, USA, 2014.
- [8] K. Cao, Y. Liu, G. Meng y Q. Sun, «An Overview on Edge Computing Research,» *IEEE Access*, vol. 8, pp. 85714 - 85728, 2020.
- [9] I. El Naqa y M. J. Murphy, «What Is Machine Learning?,» de *Machine Learning in Radiation Oncology*, Springer, 2015, pp. 3-11.

- [10] K. Das y R. N. Behera, «A Survey on Machine Learning: Concept, Algorithms and Applications,» de *International Conference on Innovative Research in Computer and Communication Engineering*, 2017.
- [11] A. Burkov, *The Hundred-Page Machine Learning Book*.
- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel y M. Blondel, «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [13] N. Moustafa y J. Slay, «UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),» *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1-6, 2015.
- [14] I. T. Jolliffe y J. Cadima, «Principal component analysis: a review and recent developments,» *Phil. Trans. R. Soc. A*, vol. 374, nº 2065, pp. 1-16, 2016.
- [15] J. R. Maria Navin y R. Pankaja, «Performance Analysis of Text Classification Algorithms using Confusion Matrix,» *International Journal of Engineering and Technical Research*, vol. 6, nº 4, pp. 75-78, 2016.